

# Hostel Room Swapping Service

A Multi-Party Hostel Room Exchange Matching Service

Project Report — Prototype Stage, UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

**Thapar Institute of Engineering and Technology**

Aabhas Khandelwal, CSED<sup>1</sup>

Aakash Chawla, CSED<sup>2</sup>

Joy Soni, CSED<sup>3</sup>

September 7, 2026

<sup>1</sup>Roll No: 1024030673, Email: [akhandelwal2\\_be24thapar.edu](mailto:akhandelwal2_be24thapar.edu)

<sup>2</sup>Roll No: 1024031097, Email: [achawla1\\_be24thapar.edu](mailto:achawla1_be24thapar.edu)

<sup>3</sup>Roll No: 1024030674, Email: [jsoni\\_be24thapar.edu](mailto:jsoni_be24thapar.edu)

# Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>3</b> |
| 1.1      | Background and Context . . . . .                      | 3        |
| 1.2      | Problem Statement . . . . .                           | 3        |
| 1.3      | Project Objectives . . . . .                          | 4        |
| <b>2</b> | <b>Methodology and Design</b>                         | <b>5</b> |
| 2.1      | System Architecture . . . . .                         | 5        |
| 2.1.1    | Web Interface . . . . .                               | 6        |
| 2.1.2    | FastAPI Backend . . . . .                             | 6        |
| 2.1.3    | Bed-Slot Modelling . . . . .                          | 6        |
| 2.2      | Preference Scoring . . . . .                          | 6        |
| 2.3      | Matching Algorithm . . . . .                          | 6        |
| 2.3.1    | Assignment Optimisation . . . . .                     | 7        |
| 2.3.2    | Alternative Generation . . . . .                      | 7        |
| 2.3.3    | Swap Feasibility . . . . .                            | 7        |
| 2.3.4    | Alternatives Considered . . . . .                     | 7        |
| 2.4      | Approval Workflow . . . . .                           | 7        |
| 2.5      | Persistence . . . . .                                 | 7        |
| 2.6      | Technical Stack . . . . .                             | 7        |
| <b>3</b> | <b>Implementation and Evaluation</b>                  | <b>9</b> |
| 3.1      | Implementation . . . . .                              | 9        |
| 3.1.1    | Round Lifecycle . . . . .                             | 9        |
| 3.1.2    | Proposal Lifecycle . . . . .                          | 9        |
| 3.2      | Evaluation Methodology . . . . .                      | 10       |
| 3.3      | Evaluation Metrics . . . . .                          | 10       |
| 3.3.1    | Cohort Preference Satisfaction . . . . .              | 10       |
| 3.3.2    | Exact-Room, Floor, and Direction Fulfilment . . . . . | 10       |
| 3.3.3    | Participants Left Worse Off . . . . .                 | 10       |
| 3.3.4    | Longest Simultaneous-Approval Chain . . . . .         | 10       |
| 3.4      | Experimental Configurations . . . . .                 | 10       |
| 3.5      | Results . . . . .                                     | 11       |
| 3.6      | Analysis of Results . . . . .                         | 11       |
| 3.6.1    | Match Quality Is Unaffected by $K$ . . . . .          | 11       |
| 3.6.2    | Executability Is Not . . . . .                        | 11       |
| 3.6.3    | Zero Worse Off, Confirmed on Real Data . . . . .      | 11       |
| 3.7      | Comparative Analysis . . . . .                        | 11       |
| 3.8      | Software Engineering Process . . . . .                | 11       |

|          |                           |           |
|----------|---------------------------|-----------|
| <b>4</b> | <b>Conclusion</b>         | <b>13</b> |
| 4.1      | Summary of Work . . . . . | 13        |
| 4.2      | Key Findings . . . . .    | 13        |
| 4.3      | Limitations . . . . .     | 13        |
| 4.4      | Future Work . . . . .     | 14        |
| 4.5      | Final Remarks . . . . .   | 14        |

# Chapter 1

## Introduction

### 1.1 Background and Context

Hostel room allotment assigns each student one room for the semester, but that allotment is fixed once made. A student who ends up on the wrong floor, facing the wrong direction, or away from friends has no structured way to change it. The informal remedy — asking around, posting in a group chat — only ever surfaces a handful of offers, and a workable trade almost always needs more than two people.

A direct one-to-one swap requires two students to each want exactly what the other already has. That coincidence is rare. What usually does exist is a longer exchange: student A takes B's room, B takes C's, and C takes A's, leaving everyone better off. No single student can see that chain from inside their own conversation, because seeing it requires knowing every other student's current room and every other student's preference at once.

MemoryLane-style content retrieval systems solve an analogous visibility problem for information; this project solves it for room allocation. The service holds every student's current room and every student's stated preference centrally, computes the exchange cycles hidden inside that data, and presents them back as concrete, executable swaps.

### 1.2 Problem Statement

Framed as requirements, a hostel room exchange service should:

1. Accept a student's preference for a specific room, with a defined fallback — floor, then direction — when that exact room cannot be obtained.
2. Compute, from every participant's current room and stated preference simultaneously, the assignment of maximum total satisfaction across the group, not the loudest individual request.
3. Generate genuinely different alternative assignments, not one answer marginally degraded, so that a shorter and more easily approved exchange can be offered alongside the mathematically optimal one.
4. Guarantee that no participant in an offered exchange ends up worse off than their starting room, as a structural property of the system rather than an incidental outcome.
5. Require every member of an exchange chain to individually approve it before any room actually changes, and ensure one member's refusal cannot silently affect any other exchange running in the same round.
6. Preserve a complete, non-rewritable history of who held which room and when, so that any executed swap is auditable after the fact.

## 1.3 Project Objectives

1. **Closed-pool assignment.** Model the swap problem as a permutation of students over the rooms they already collectively hold, so that swap chains are not searched for separately but fall directly out of the assignment’s cycle structure.
2. **Optimal and alternative assignment.** Solve for the single best assignment using a proven combinatorial algorithm, and generate a bounded number of alternative assignments so that a shorter, more executable chain can be preferred when it exists at no cost to overall match quality.
3. **Feasibility filtering.** Discard any candidate exchange that would leave a participant worse off, independent of whatever the raw optimisation objective would otherwise select.
4. **A real approval workflow.** Model proposal, approval, execution, rejection and expiry as an explicit state machine, separate from the state machine that governs a matching round itself.
5. **Persistent, auditable storage.** Store allocation history as an append-only log in a managed relational database, with database-level enforcement of that guarantee.
6. **Measurable evaluation.** Evaluate the engine against a reproducible cohort using metrics that describe the whole participant group, not only the students who moved.

## Chapter 2

# Methodology and Design

### 2.1 System Architecture

The service is organised as a pipeline connecting a browser-based interface, a stateless API layer, a pure matching engine, and a managed relational database.

The major components are:

1. Admin and student web pages
2. FastAPI backend and session-based authentication
3. Preference and bed-slot modelling
4. Cost-matrix construction and scoring
5. Hungarian-algorithm assignment
6. Murty's-algorithm alternative generation
7. Cycle decomposition and Pareto feasibility filtering
8. Approval (proposal) state machine
9. PostgreSQL persistence with append-only allocation history

The registration pipeline is:

**Student Login → Round Enrolment → Weighted Priorities → Stored Preference Set**

The matching pipeline is:

**Round Locked → Cost Matrix → Hungarian Assignment → Murty  $K$ -Best → Cycle Decomposition → Pareto Filter → Proposals Offered**

Separating registration from matching, and matching from approval, means each stage can be developed, tested, and reasoned about independently: the round records what the engine found to be *possible*, and the proposal state machine records what the participants actually *agreed to*.

### 2.1.1 Web Interface

Two role-scoped web views sit in front of the service: a student view for enrolling in a round, setting priorities, and responding to proposed exchanges, and a warden (admin) view for creating and locking a round for a given hostel. Both are served as plain HTML/JavaScript pages by the same backend that exposes the API, so no separate frontend deployment is required at this stage. A migration to a React-based frontend is planned; because the pages consume the same JSON API a script or a framework component would, that migration does not require any change to the backend or matching engine.

### 2.1.2 FastAPI Backend

The backend exposes 27 endpoints across four groups: authentication (login, logout, session lookup), student actions (enrol, set priorities, view results, offer or respond to a proposal), admin actions (create, open, and lock a round; declare room-type inventory), and two diagnostic endpoints (`/api/schema`, which reports the live database structure, and `/api/demo`, a fixed-cohort endpoint used for reproducible evaluation, described in Section 3.2). Every write endpoint that requires a specific role is rejected at the backend if the authenticated session does not hold that role.

### 2.1.3 Bed-Slot Modelling

A student’s stated preferences — hostel, floor, direction, room type, washroom — are properties of a *room*. Assignment, however, has to happen over a *one-to-one* structure for the Hungarian algorithm to apply at all. A shared room with three beds is not one assignable unit; it is three. The system therefore assigns students to individual bed slots rather than rooms directly, with each room expanding into as many slots as its declared capacity. This keeps the assignment problem a clean permutation — as many slots as participating students — while still allowing shared rooms and mutual roommate requests to be expressed, without which the whole Hungarian/Murty formulation would not apply.

## 2.2 Preference Scoring

Each student’s satisfaction with a candidate room is the fraction of their stated preference weight that room satisfies. Preferences are distinguished as hard or soft: an unmet soft preference lowers the score, while an unmet hard preference removes the room from consideration entirely rather than merely penalising it. In the current evaluation cohort, priorities follow a fixed fallback ladder — a named room is worth more than its floor, which is worth more than its direction — so that a student who cannot obtain their exact room still scores well for landing near where their friends are.

## 2.3 Matching Algorithm

The pool of participants in a round is closed: the only rooms available are the ones the participants already collectively hold. Consequently, any solution is a permutation of students over slots, and every permutation decomposes uniquely into disjoint cycles. Swap chains are therefore not searched for separately from the assignment — they are already present inside it, and the system reads them out.

### 2.3.1 Assignment Optimisation

The Hungarian algorithm, via `scipy.optimize.linear_sum_assignment` [kuhn1955hungarian; virtanen2020scipy], solves the students-by-slots cost matrix for the assignment of maximum total satisfaction in  $O(n^3)$ .

### 2.3.2 Alternative Generation

Murty’s algorithm [murty1968ranking] enumerates the  $K$  best assignments by repeatedly forbidding one edge of an already-found assignment and re-solving. This is a correctness requirement rather than a convenience feature: the single mathematically optimal assignment frequently demotes one participant to benefit several others, which the feasibility filter (Section 2.3.3) then rejects outright — the alternatives are where an *executable* exchange is actually found.

### 2.3.3 Swap Feasibility

Each candidate assignment is decomposed into its disjoint exchange cycles. Because the cycles are disjoint, they execute independently: a single student withdrawing from one cycle provably cannot affect any other cycle in the same assignment. A Pareto filter then discards any cycle in which a participant would end up worse off than their current room, keeping only chains that every member has a rational reason to accept.

### 2.3.4 Alternatives Considered

Gale–Shapley stable matching [gale1962college] was considered and rejected: it assumes two-sided preferences, where both sides rank each other, and rooms hold no preference over students, so half the algorithm has no input. Top Trading Cycles [shapley1974cores] fits the exchange shape of the problem directly, but operates over a strict ranked list of items and cannot represent preferences that are weighted across several attributes at once, as ours are.

## 2.4 Approval Workflow

A candidate exchange chain is issued as a proposal, not applied immediately. Every member of the chain must individually approve it; a single rejection kills only that one proposal. A round moves through its own lifecycle — draft, open, locked, running, completed — independently of the approval state of the proposals it produced, which is what allows a round to be complete (the engine has run and produced its answer) while individual proposals inside it are still being negotiated.

## 2.5 Persistence

The schema spans 14 tables in a managed PostgreSQL database (Supabase): students and sessions; hostels and their declared room-type inventory; rooms and bed slots; the allocation history; preference sets; swap rounds and enrolments; and the proposal and swap-chain-member tables backing the approval workflow. Allocation history is append-only, enforced by a database trigger rather than by application-level convention, so a defect in a future client cannot silently rewrite who held which room and when.

## 2.6 Technical Stack



| Component              | Technology                                   |
|------------------------|----------------------------------------------|
| Programming language   | Python                                       |
| Backend framework      | FastAPI                                      |
| Assignment solver      | SciPy ( <code>linear_sum_assignment</code> ) |
| Alternative generation | Murty's algorithm (in-house, over SciPy)     |
| Database               | PostgreSQL (Supabase, managed)               |
| Authentication         | Session cookies, server-side session table   |
| Testing                | pytest                                       |
| CI/CD                  | GitHub Actions                               |
| Frontend (current)     | Server-rendered HTML / JavaScript            |
| Frontend (planned)     | React                                        |

Table 2.1: Technical Stack

## Chapter 3

# Implementation and Evaluation

### 3.1 Implementation

The matching engine is implemented as a pure Python package with no framework or database import anywhere in it, so it can be exercised and tested with no infrastructure running at all.

- `scoring.py` — preference weighting and match calculation.
- `matrix.py` — students-by-slots cost matrix construction.
- `solver.py` — Hungarian-algorithm assignment.
- `kbest.py` — Murty’s  $K$ -best alternative search.
- `cycles.py` — cycle decomposition into exchange chains.
- `feasibility.py` — the Pareto feasibility filter.
- `ranking.py` — candidate scoring and selection.
- `proposal.py` — the approval state machine.
- `rounds.py` — the round lifecycle and orchestration of the pipeline above.
- `db.py` — the only module that speaks SQL; loads and persists the same data shapes the engine consumes.
- `auth.py` / `api.py` — session authentication and the FastAPI route layer.

#### 3.1.1 Round Lifecycle

A round for one hostel moves through `draft` → `open` → `locked` → `running` → `completed`. Locking a round is a one-way transition: it closes registration and triggers the matching pipeline in the same action, after which the round cannot return to `open`.

#### 3.1.2 Proposal Lifecycle

Each offered exchange chain moves through `proposed` → `approved` → `executed`, or exits via `rejected` or `expired`. Execution appends new allocation rows for every member of the chain; no existing row is ever modified.

## 3.2 Evaluation Methodology

No public benchmark exists for hostel room-swap matching, so the engine is evaluated against a reproducible synthetic cohort rather than an external dataset. A pool of 30 students, one per room, in a single hostel, was drawn once from a seeded random generator and frozen, so evaluation results are exactly reproducible run to run rather than depending on which random cohort happened to be sampled that day. Each student’s priorities follow the room/floor/direction fallback ladder described in Section 2.1.3, with no preference toward the hostel itself, since every student in the cohort already lives there.

The cohort is exercised through the service’s own `/api/demo` endpoint, which runs the identical matching pipeline used by a real locked round, so the benchmark evaluates the engine’s real code path rather than a simulation of it.

## 3.3 Evaluation Metrics

### 3.3.1 Cohort Preference Satisfaction

The primary metric is the mean weighted match score across *every* participant in the cohort, computed before and after one matching round — not only across the students who moved. This is a deliberate choice: averaging over movers alone would rank a two-person swap in which both reach 100% ahead of an eleven-person rotation that lifts the entire cohort, which is not the outcome the service exists to produce.

### 3.3.2 Exact-Room, Floor, and Direction Fulfilment

Secondary metrics report the proportion of the cohort receiving the exact room they named, the proportion receiving at least their requested floor, and the proportion receiving at least their requested direction — the three rungs of the fallback ladder.

### 3.3.3 Participants Left Worse Off

This count must be zero by construction, since the Pareto filter (Section 2.3.3) discards any chain violating it; it is reported as a measured value rather than assumed, to confirm the filter behaves as specified on real data rather than only in unit tests.

### 3.3.4 Longest Simultaneous-Approval Chain

The size of the largest exchange cycle in the offered assignment, reported because it is the practical execution risk of a chain: every additional member is one more independent approval that must be obtained before the whole chain can execute.

## 3.4 Experimental Configurations

Two configurations of the same pipeline were evaluated on the frozen cohort, differing only in  $K$ , the number of alternative assignments Murty’s algorithm explores before the best candidate is selected:

1.  $K = 1$ : only the single mathematically optimal assignment is considered.
2.  $K = 80$  (**production**): the full Murty’s search is run, and the assignment maximising cohort satisfaction is selected from among the alternatives, breaking ties toward a shorter longest chain.

## 3.5 Results

## 3.6 Analysis of Results

### 3.6.1 Match Quality Is Unaffected by $K$

Cohort satisfaction, the count of students moved, and the count receiving their exact requested room are identical at  $K = 1$  and  $K = 80$ . On this cohort, exploring alternative assignments does not trade away any match quality at all.

### 3.6.2 Executability Is Not

The longest chain requiring simultaneous approval falls from 28 students to 11 between the two configurations, at no cost measured above. A chain of 28 means 28 independent people must each agree before anything executes, and a single hold-out anywhere in that chain blocks the entire exchange. A chain of 11 carries proportionally less of that risk, and because the underlying cycles are disjoint by construction (Section 2.3.3), a shorter maximum chain generally also means the assignment is expressible as several smaller independent chains rather than one large one.

### 3.6.3 Zero Worse Off, Confirmed on Real Data

The Pareto guarantee is a property proved of the filter’s logic in unit tests; Table 3.1 confirms it also holds when the filter is exercised against a full 30-student assignment rather than a small hand-constructed case.

## 3.7 Comparative Analysis

These results are the concrete justification for including Murty’s algorithm in the design at all: a single Hungarian solve would have been sufficient if match quality were the only concern, but it is not — an assignment that cannot realistically be approved is not a useful answer, and  $K$ -best search is what turns an optimal-on-paper assignment into one that is also practical to execute.

## 3.8 Software Engineering Process

The full automated test suite — 127 tests — runs in approximately two seconds with no live database, a direct consequence of the matching engine importing no framework or database driver: unit and scenario tests exercise the pure Python pipeline directly, and only a small, isolated set of tests exercises the PostgreSQL integration path so the rest of the suite remains hermetic on a machine with no database credentials at all. GitHub Actions runs the suite on every push and pull request, across Python 3.10, 3.11, and 3.12.

Three defects surfaced only once real infrastructure was in the loop, each caught before it reached a shared environment:

- A preference criterion added to the Python model was never added to the corresponding PostgreSQL enum, which would have rejected the very first insert against a fresh database.
- A database driver installed by hand on a development machine was never declared as a project dependency, so it worked locally and failed immediately in a clean CI environment.
- A deliberate offline fallback — used so a demo cannot be broken by a database outage — initially made a genuine misconfiguration indistinguishable from a correctly working system, until the API response was made to report which data source it had actually used.

| <b>Metric</b>                                   | $K = 1$               | $K = 80$              |
|-------------------------------------------------|-----------------------|-----------------------|
| Cohort satisfaction, before $\rightarrow$ after | 11% $\rightarrow$ 77% | 11% $\rightarrow$ 77% |
| Students moved                                  | 28 / 30               | 28 / 30               |
| Received exact requested room                   | 20 / 30               | 20 / 30               |
| Participants left worse off                     | 0                     | 0                     |
| Longest simultaneous-approval chain             | 28                    | 11                    |

Table 3.1: Matching Engine Evaluation, Frozen 30-Student Cohort

| <b>Observation</b>                               | <b>Interpretation</b>                                                                                                                                   |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Satisfaction identical at $K = 1$ and $K = 80$   | Murty’s alternative search finds a more executable assignment without sacrificing overall match quality on this cohort.                                 |
| Longest chain: 28 at $K = 1$ , 11 at $K = 80$    | Far fewer simultaneous approvals are required for the same outcome, directly reducing the practical risk of the exchange collapsing before it executes. |
| 20 of 30 receive their exact requested room      | The majority of demand is satisfiable without falling back to floor or direction on this cohort.                                                        |
| 0 participants worse off, in both configurations | The feasibility filter’s guarantee held with no exception across the full assignment, not only on average.                                              |

Table 3.2: Comparison of Evaluated Configurations

# Chapter 4

## Conclusion

### 4.1 Summary of Work

The prototype implements a complete pipeline from student preference submission through to an executed room swap: a pure matching engine combining the Hungarian algorithm with Murty’s  $K$ -best alternative search and a Pareto feasibility filter; a proposal-based approval workflow layered on top of a separate round lifecycle; and a PostgreSQL-backed persistence layer with an enforced append-only allocation history. A session-authenticated web interface exposes both a student role and a warden (admin) role over the same backend.

### 4.2 Key Findings

1. **The engine finds exchanges no participant could see alone.** Multi-party chains are read directly out of a computed permutation rather than searched for separately.
2. **Alternative-assignment search reduces execution risk at no measured cost to match quality.** On the evaluated cohort, raising  $K$  cut the longest simultaneous-approval chain from 28 students to 11 while leaving cohort satisfaction unchanged.
3. **The feasibility guarantee held with zero exceptions** across the full evaluated assignment, not only in isolated unit tests.
4. **The system performs practically.** The evaluated 30-student assignment computes in well under a second, and the matching engine’s freedom from framework and database dependencies keeps the automated test suite fast enough to run on every push.
5. **Testing against real infrastructure caught real defects.** Three separate issues — a schema/model mismatch, an undeclared dependency, and a silent fallback — were each found because the prototype was exercised against a genuine database and CI pipeline early, rather than only against hand-written unit tests.

### 4.3 Limitations

- The engine has been evaluated against a seeded synthetic cohort, not yet against real student registrations for a live hostel.
- Shared (multi-bed) rooms and mutual roommate requests are supported by the bed-slot model and scoring function, but are not exercised by the current evaluation cohort, which uses one student per room so every chain reads as an unambiguous rotation.

- Proposal expiry and round cancellation exist in the state model but have not yet been demonstrated end-to-end on a live clock.
- The frontend is presently server-rendered HTML/JavaScript rather than the React frontend named in the project proposal.

## 4.4 Future Work

1. Onboard real student preference registrations for at least one hostel, replacing the seeded evaluation cohort.
2. Exercise shared rooms and mutual roommate matching end-to-end, including on the live evaluation cohort.
3. Demonstrate the reject, expiry, and round-cancellation paths live rather than only in the test suite.
4. Migrate the frontend to React against the existing, unchanged API contract.
5. Evaluate the engine at larger cohort sizes to characterise how satisfaction, chain length, and solve time scale together.

## 4.5 Final Remarks

This prototype demonstrates that a hostel’s worth of room-swap requests can be resolved as a single closed assignment problem, with the exchange chains falling directly out of that assignment’s cycle structure rather than being searched for separately. The evaluation shows that generating alternative assignments is not merely an academic completeness exercise: on the tested cohort it produced a markedly more executable outcome — an 11-person longest chain in place of a 28-person one — without giving up any measured match quality. The remaining work is primarily about exercising paths the architecture already supports against real data, rather than redesigning the underlying approach.